



EQUIPO NARANJA

ASIGNACIÓN ESCRITA III

Uso de Pilas y Colas en la Solución de Problemas

DATOS DEL EQUIPO	
Equipo	Naranja - Equipo #15
Integrantes Y Matriculas	Hugo E. Montero Fulcar Matricula 100082929
	Erasmus José Minaya Taveras Matricula 100076710
	Albert Daniel Montero Ramírez Matricula 100078731
	Luis Ángel Mora Taveras Matricula 100076432
	Fernando Enrique Mesón Acosta Matricula 100027782
Asignatura	Estructura de Datos
Institución	Universidad Abierta para Adultos (UAPA)
Fecha	29 de mayo 2026

1. INTRODUCCIÓN

Las estructuras de datos son uno de los pilares fundamentales de la programación y la ingeniería de software. Entre las más importantes se encuentran las Pilas (Stack) y las Colas (Queue), estructuras lineales que permiten organizar y procesar información de manera eficiente bajo principios bien definidos.

Una Pila sigue el principio LIFO (Last In, First Out): el último elemento en ingresar es el primero en salir. Esto la convierte en la estructura ideal para modelar comportamientos como el historial de navegación web y la evaluación de expresiones matemáticas en notación postfija.

Una Cola sigue el principio FIFO (First In, First Out): el primer elemento en ingresar es el primero en salir. Esto la hace perfecta para simular sistemas de procesamiento ordenado, como colas de empaque en almacenes o carritos de compras en línea.

El presente informe documenta la implementación en C++ con Paradigma Orientado a Objetos (POO) de los cuatro ejercicios del Equipo Naranja, incluyendo el análisis de complejidad algorítmica en notación Big-O de cada función implementada. Todos los programas utilizan `getline()` para la lectura de entrada, garantizando compatibilidad con entornos de compilación en línea como OnlineGDB.

2. EJERCICIOS CON PILAS

Ejercicio 1 — Sistema de Navegación Web (Atrás/Adelante)

2.1 Descripción del problema

Se implementa un simulador de navegador web que permite al usuario visitar páginas, navegar hacia atrás y hacia adelante. Se utilizan dos pilas: una para el historial hacia atrás (historialAtras) y otra para el historial hacia adelante (historialAdelante).

2.2 Diseño de la solución — Clase Navegador

La clase Navegador encapsula el estado del navegador en tres atributos privados y expone cuatro métodos públicos. Toda la entrada del usuario se procesa con `getline()` e `istringstream` para evitar conflictos en el buffer de entrada.



ej1_pilas_navegacion.cpp

2.3 Lógica de cada operación

- `visitar(url)`: Guarda la página actual en `historialAtras`, limpia `historialAdelante` (ya no tiene sentido avanzar tras una nueva visita), y establece la nueva URL como página actual. Complejidad $O(1)$.
- `atras()`: Mueve la página actual a `historialAdelante` y recupera el tope de `historialAtras` como nueva página actual. Complejidad $O(1)$.
- `adelante()`: Mueve la página actual a `historialAtras` y recupera el tope de `historialAdelante`. Complejidad $O(1)$.
- `mostrar()`: Copia ambas pilas para iterarlas sin destruirlas e imprime el estado completo. Complejidad $O(n)$.

2.4 Ejemplo de ejecución (entrada del enunciado)

```
> visitar google.com      -> Visitando: google.com
> visitar uapa.edu.do     -> Visitando: uapa.edu.do
> visitar github.com      -> Visitando: github.com
> atras                   -> Pagina actual: uapa.edu.do
> atras                   -> Pagina actual: google.com
> adelante                -> Pagina actual: uapa.edu.do
> mostrar
Pagina actual: uapa.edu.do
Historial atras: [google.com]
Historial adelante: [github.com]
```

2.5 Screenshot ejecutando código

```
=== SISTEMA DE NAVEGACION WEB ===
Comandos: visitar <url> | atras | adelante | mostrar | salir
=====

> Visitar Google.com
Comando no reconocido: Visitar
> visitar Google.com
Visitando: Google.com
> mostrar
Pagina actual: Google.com
Historial atras: []
Historial adelante: []
> visitar UAPA.com
Visitando: UAPA.com
> visitar Pixiv.com
Visitando: Pixiv.com
> Mostrar
Comando no reconocido: Mostrar
> mostrar
Pagina actual: Pixiv.com
Historial atras: [Google.com, UAPA.com]
Historial adelante: []
> atras
Pagina actual: UAPA.com
> mostrar
Pagina actual: UAPA.com
Historial atras: [Google.com]
Historial adelante: [Pixiv.com]
> █
```

2.6 Análisis de Complejidad Algorítmica

Función	Mejor caso	Caso medio	Peor caso	Observación
visitar(url)	$O(1)$	$O(1)$	$O(1)$	Push/pop en pila: tiempo constante
atras()	$O(1)$	$O(1)$	$O(1)$	Pop histAtras, push histAdelante
adelante()	$O(1)$	$O(1)$	$O(1)$	Pop histAdelante, push histAtras
mostrar()	$O(n)$	$O(n)$	$O(n)$	n = páginas en el historial
GENERAL	$O(n)$	$O(n)$	$O(n)$	Dominado por mostrar()

Ejercicio 2 — Calculadora Postfija (Notación Polaca Inversa)

2.7 Descripción del problema

Se implementa una calculadora que evalúa expresiones en notación postfija (RPN). En esta notación los operadores aparecen después de sus operandos, eliminando la necesidad de paréntesis y de reglas de precedencia. Se usa una pila de operandos para el procesamiento.

2.8 Diseño de la solución — Clase CalculadoraPostfija

La clase encapsula tres métodos privados de apoyo y un método público evaluar(). La entrada completa se lee con getline() y se tokeniza con istringstream, procesando cada token uno a uno.



ej2_pilas_calculador
a_postfija.cpp

2.9 Algoritmo de evaluación (Shunting-Yard simplificado)

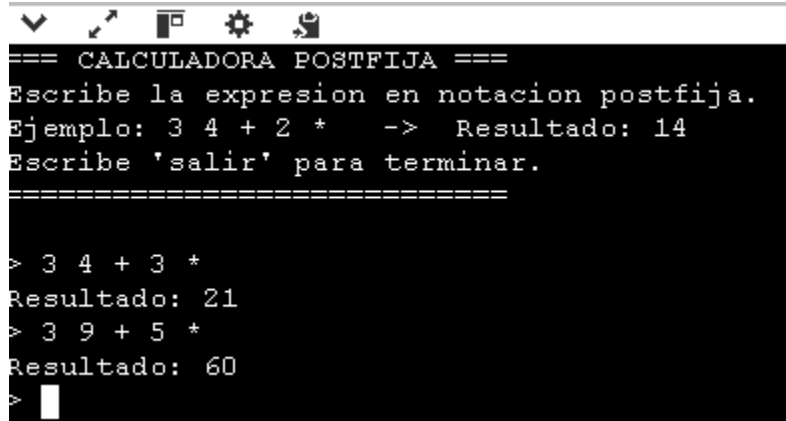
- Si el token es un número → push a la pila de operandos.
- Si el token es un operador → pop dos operandos (b luego a), aplicar la operación, push el resultado.
- Si al finalizar la pila contiene exactamente 1 elemento → ese es el resultado final.
- En cualquier otro escenario → la expresión es inválida y se notifica al usuario.

2.10 Ejemplo de ejecución (entrada del enunciado)

```
> 3 4 + 2 *
Paso 1: push 3          -> Pila: [3]
Paso 2: push 4          -> Pila: [3, 4]
Paso 3: '+' pop 4,3     -> 3+4=7, push 7 -> Pila: [7]
Paso 4: push 2          -> Pila: [7, 2]
Paso 5: '*' pop 2,7     -> 7*2=14, push 14 -> Pila: [14]
Resultado: 14

> 3 +
Paso 1: push 3          -> Pila: [3]
Paso 2: '+' -> solo 1 operando disponible
Error: expresion invalida (faltan operandos).
```

2.11 Screenshot ejecutando código



```

=== CALCULADORA POSTFIJA ===
Escribe la expresion en notacion postfija.
Ejemplo: 3 4 + 2 * -> Resultado: 14
Escribe 'salir' para terminar.
=====

> 3 4 + 3 *
Resultado: 21
> 3 9 + 5 *
Resultado: 60
> 
```

2.12 Análisis de Complejidad Algorítmica

Función	Mejor caso	Caso medio	Peor caso	Observación
esOperador(token)	$O(1)$	$O(1)$	$O(1)$	Comparación de 4 valores posibles
esNumero(s)	$O(1)$	$O(n)$	$O(n)$	n = longitud del string del token
operar(a, b, op)	$O(1)$	$O(1)$	$O(1)$	Operación aritmética básica
evaluar(expr)	$O(n)$	$O(n)$	$O(n)$	n = número de tokens en la expresión
GENERAL	$O(n)$	$O(n)$	$O(n)$	Cada token se procesa exactamente una vez

3. EJERCICIOS CON COLAS

Ejercicio 1 — Simulador de Cola de Empaque en un Almacén

3.1 Descripción del problema

Se implementa un sistema que gestiona una cola de cajas en un almacén. Cada caja tiene un ID y una descripción. Las cajas se procesan en el orden en que llegan (FIFO), simulando una línea de empaque real donde lo primero que entra es lo primero en salir.

3.2 Diseño de la solución — Struct Caja y Clase ColaEmpaque

Se define la estructura Caja para agrupar los datos de cada elemento, y la clase ColaEmpaque que encapsula una `queue<Caja>` de la STL de C++. La entrada se procesa completamente con `getline()` e `istringstream`.



3.3 Lógica de cada operación

- `agregar(id, desc)`: Crea un objeto Caja y lo añade al final de la cola con `push()`. Tiempo $O(1)$.
- `procesar()`: Extrae la caja del frente con `front()` y `pop()`, imprimiendo el mensaje de procesamiento. Tiempo $O(1)$.
- `verCola()`: Genera una copia de la cola para iterar sin modificar la original, mostrando posición, ID y descripción de cada caja. Tiempo $O(n)$.

3.4 Ejemplo de ejecución (entrada del enunciado)

```
> agregar C001 Articulos electronicos    -> Caja C001 agregada: Articulos
electronicos
> agregar C002 Ropa deportiva            -> Caja C002 agregada: Ropa deportiva
> agregar C003 Alimentos                 -> Caja C003 agregada: Alimentos
> verCola
--- Cola de Empaque ---
1. [C001] Articulos electronicos
2. [C002] Ropa deportiva
3. [C003] Alimentos
Total: 3 caja(s)
> procesar    -> Procesando caja C001: Articulos electronicos
> verCola
1. [C002] Ropa deportiva
2. [C003] Alimentos
Total: 2 caja(s)
```

3.6 Screenshot ejecutando código

```
input
=== COLA DE EMPAQUE ===
Comandos:
  agregar <id> <descripcion>
  procesar
  verCola
  salir
=====
> agregar C001 Tarjetas Graficas
Caja C001 agregada: Tarjetas Graficas
> agregar C002 Procesadores
Caja C002 agregada: Procesadores
> agregar C003 Memoria RAM
Caja C003 agregada: Memoria RAM
> VerCola
Comando no reconocido: VerCola
> verCola
--- Cola de Empaque ---
1. [C001] Tarjetas Graficas
2. [C002] Procesadores
3. [C003] Memoria RAM
Total: 3 caja(s)
-----
> procesar
Procesando caja C001: Tarjetas Graficas
> verCola
--- Cola de Empaque ---
1. [C002] Procesadores
2. [C003] Memoria RAM
Total: 2 caja(s)
-----
> 
```

3.7 Análisis de Complejidad Algorítmica

Función	Mejor caso	Caso medio	Peor caso	Observación
agregar(id, desc)	$O(1)$	$O(1)$	$O(1)$	enqueue al final de la cola
procesar()	$O(1)$	$O(1)$	$O(1)$	dequeue del frente de la cola
verCola()	$O(n)$	$O(n)$	$O(n)$	n = número de cajas en la cola
GENERAL	$O(n)$	$O(n)$	$O(n)$	Dominado por verCola()

Ejercicio 2 — Carrito de Compras en Línea

3.8 Descripción del problema

Se implementa el proceso de checkout de un carrito de compras en línea. Cada producto tiene nombre y cantidad. Los pagos se procesan en orden de llegada (FIFO), simulando el comportamiento de un sistema de comercio electrónico donde los primeros productos agregados son los primeros en procesarse.

3.9 Diseño de la solución — Struct Producto y Clase CarritoCompras

Se define la estructura Producto y la clase CarritoCompras con una `queue<Producto>`. Como en todos los ejercicios, la lectura de entrada usa `getline()` e `istringstream` para compatibilidad total con OnlineGDB.



3.8 Lógica de cada operación

- `agregar(nombre, cantidad)`: Valida que la cantidad sea positiva, crea un `Producto` y lo añade al final de la cola. Tiempo $O(1)$.
- `pagar()`: Extrae el primer producto de la cola e imprime el mensaje de procesamiento de pago. Tiempo $O(1)$.
- `mostrarCarrito()`: Copia la cola para mostrar todos los productos sin modificarla. Muestra nombre y cantidad de cada uno. Tiempo $O(n)$.

3.9 Ejemplo de ejecución (entrada del enunciado)

```
> agregar Leche 2      -> Agregado: 2 Leche
> agregar Pan 3        -> Agregado: 3 Pan
> agregar Jugo 1       -> Agregado: 1 Jugo
> mostrarCarrito
--- Carrito de Compras ---
1. 2x Leche
2. 3x Pan
3. 1x Jugo
Productos en carrito: 3
> pagar -> Procesando pago de 2 Leche
> mostrarCarrito
1. 3x Pan
2. 1x Jugo
Productos en carrito: 2
```

3.10 Screenshot ejecutando código

```
=== CARRITO DE COMPRAS ===
Comandos:
  agregar <nombre> <cantidad>
  pagar
  mostrarCarrito
  salir
=====

> agregar LinLan 75
Agregado: 75 LinLan
> agregar CorsairARGB 150
Agregado: 150 CorsairARGB
> agregar Thermaright 20
Agregado: 20 Thermaright
> mostrarCarrito
--- Carrito de Compras ---
1. 75x LinLan
2. 150x CorsairARGB
3. 20x Thermaright
Productos en carrito: 3
-----
> pagar
Procesando pago de 75 LinLan
> mostrarCarrito
--- Carrito de Compras ---
1. 150x CorsairARGB
2. 20x Thermaright
Productos en carrito: 2
-----
>
```

3.11 Análisis de Complejidad Algorítmica

Función	Mejor caso	Caso medio	Peor caso	Observación
agregar(nombre, cant)	$O(1)$	$O(1)$	$O(1)$	enqueue al final de la cola
pagar()	$O(1)$	$O(1)$	$O(1)$	dequeue del frente de la cola
mostrarCarrito()	$O(n)$	$O(n)$	$O(n)$	n = productos en el carrito
GENERAL	$O(n)$	$O(n)$	$O(n)$	Dominado por mostrarCarrito()

4. RESUMEN DE COMPLEJIDADES

La siguiente tabla consolida las complejidades generales de los cuatro ejercicios implementados:

#	Ejercicio	Estructura	Complejidad	Función dominante
1	Sistema de Navegación Web	Pila (LIFO)	$O(n)$	mostrar()
2	Calculadora Postfija	Pila (LIFO)	$O(n)$	evaluar()
3	Cola de Empaque	Cola (FIFO)	$O(n)$	verCola()
4	Carrito de Compras	Cola (FIFO)	$O(n)$	mostrarCarrito()

5. CONCLUSIONES

A través del desarrollo de los cuatro ejercicios del Equipo Naranja se pudieron verificar en la práctica las siguientes observaciones:

- Las Pilas (LIFO) fueron la elección natural para el historial de navegación web y la evaluación de expresiones postfijas, donde el orden inverso de procesamiento es esencial para el correcto funcionamiento.
- Las Colas (FIFO) resultaron perfectas para la cola de empaque y el carrito de compras, donde el orden de llegada debe respetarse estrictamente para garantizar un procesamiento justo y ordenado.
- Las operaciones fundamentales de ambas estructuras — push, pop, enqueue, dequeue — tienen complejidad $O(1)$, lo que las hace altamente eficientes para los escenarios implementados.
- La complejidad general de cada programa está determinada por las operaciones de visualización y recorrido, que tienen complejidad $O(n)$ donde n es el número de elementos almacenados.
- El Paradigma Orientado a Objetos (POO) permitió encapsular correctamente el estado y comportamiento de cada estructura, mejorando la legibilidad, el mantenimiento y la extensibilidad del código.
- El uso exclusivo de `getline()` para la lectura de entrada garantizó la compatibilidad completa con el compilador en línea OnlineGDB, eliminando los problemas de buffer que causaban el bloqueo de la entrada interactiva.
- La STL de C++ (`stack<T>` y `queue<T>`) simplificó la implementación al proveer estructuras optimizadas, permitiendo enfocar el desarrollo en la lógica del problema y no en detalles de bajo nivel.

En conclusión, las pilas y colas son estructuras de datos esenciales que, correctamente aplicadas, ofrecen soluciones elegantes y eficientes a una amplia variedad de problemas computacionales del mundo real.